

Завдання 1. У цій роботі я створила функцію `print_card()` без параметрів для друку своїх даних і викликала її тричі. Потім написала функцію `print_card_args()` із параметром за замовчуванням для групи та протестувала її виклик різними способами: за порядком, за іменами та їх міксом. Також перевірила помилки: якщо не передати всі деталі — виникає `TypeError`, а якщо поставити звичайне значення після іменованого — синтаксична помилка `SyntaxError` вибиває ще до запуску коду.

```

1  def print_card():
2      print("Каріна Батура, IT-31, 2009")
3
4
5  print_card()
6  print_card()
7  print_card()
8
9
10 def print_card_args(name, surname, year, group="IT-31"):
11     print(f"{name} {surname}, {group}, {year}")
12
13
14 print_card_args("Каріна", "Батура", 2009, "IT-31")
15 print_card_args(surname="Батура", year=2009, name="Каріна", group="IT-31")
16 print_card_args("Каріна", "Батура", group="IT-31", year=2009)
17 print_card_args("Каріна", "Батура", 2009)

```

```

C:\Users\Python\Python313\python
Каріна Батура, IT-31, 2009
Каріна Батура, IT-31, 2009
Каріна Батура, IT-31, 2009

```

Завдання 2. У цьому завданні я створила дві функції для обчислення віку за роком народження, щоб зрозуміти різницю між `print` та `return`.

Перша функція `print_age()` просто виводить вік на екран і віддає `None`, тому її результат не можна використати далі. Друга функція `get_age()` повертає число через `return`, що дозволило мені порахувати вік у місяцях, тижнях та для 2030 року. Також я додала перевірку на некоректний рік і переконалася, що після виконання `return` функція одразу зупиняється, а весь код нижче просто ігнорується.

```

1  #Батури Каріні IT - 31
2  y = 2009
3
4  def print_age(year, current_year=2026):
5      print(f"Age: {current_year - year}")
6
7  def get_age(year, current_year=2026):
8      if year > current_year or year < 0:
9          return -1
10     return current_year - year
11     print("after return")
12
13
14 print_age(y)
15 print(f"print_age returned: {print_age(y)}")
16
17 age = get_age(y)
18 print(f"Age from get_age: {age}")
19 print(f"Age in months: {age * 12}")
20 print(f"Age in weeks: {age * 52}")
21
22 print(f"Age in 2030: {get_age(y, 2030)}")
23
24 print(f"Invalid year 3000 gives: {get_age(3000)}")

```

```

Age: 17
Age: 17
print_age returned: None
Age from get_age: 17
Age in months: 204
Age in weeks: 884
Age in 2030: 21
Invalid year 3000 gives: -1

```

Завдання 3. У цьому завданні я створила бібліотеку функцій для роботи з текстом, які формують ініціали, рахують символи та голосними, а також розвертають рядок циклом. В основній частині програми я вивела дані про своє прізвище, порахувала приголосні через різницю довжини й голосних та використала параметри за замовчуванням. Також я додала анотації типів і docstring, вивівши їх у консоль через системні атрибути `__doc__` та `__annotations__`.

```
#Батури Каріни ІТ - 31
def get_initials(name: str, surname: str) -> str:
    return f"{name[0].upper()}.{surname[0].upper()}."

def count_letters(text: str, letter: str = "a") -> int:
    """Return how many times letter occurs in text."""
    count = 0
    for char in text.lower():
        if char == letter.lower():
            count += 1
    return count

def count_vowels(text: str) -> int:
    vowels = "aeiouy"
    count = 0
    for char in text.lower():
        if char in vowels:
            count += 1
    return count

def reverse_text(text: str) -> str:
    reversed_str = ""
    for char in text:
        reversed_str = char + reversed_str
    return reversed_str

name = "Karina"
surname = "Batura"
initials = get_initials(name, surname)
letters = count_letters(surname)
vowels = count_vowels(surname)
consonants = letters - vowels
reversed_surname = reverse_text(surname)
docstring = count_letters.__doc__
annotations = count_letters.__annotations__

c = len(surname)
vowels = count_vowels(surname)
consonants = c - vowels

print(f"Letters in surname: {c}")
print(f"Vowels: {vowels}, consonants: {consonants}")

for v in "aeiou":
    print(f"{v}: {count_letters(surname, letter=v)}")

print(f"Default letter 'a': {count_letters(surname)}")
print(f"Reversed surname: {reverse_text(surname)}")
print(f"Docstring: {count_letters.__doc__}")
print(f"Annotations: {count_letters.__annotations__}")

name = "Karina"
initials = get_initials(name, surname)
print(initials)
letters = count_letters(surname)
vowels = count_vowels(surname)
consonants = letters - vowels
reversed_surname = reverse_text(surname)
docstring = count_letters.__doc__
annotations = count_letters.__annotations__
print(f"Letters in surname: {letters}")
print(f"Vowels: {vowels}, consonants: {consonants}")
print(f"Default letter 'a': {count_letters(surname, letter='a')}")
print(f"Reversed surname: {reverse_text(surname)}")
print(f"Docstring: {count_letters.__doc__}")
print(f"Annotations: {count_letters.__annotations__}")
```

Завдання 4. У цій лабораторній роботі я засвоїла принципи модульного програмування в Python, розбравшись із різницею між виводом на екран через `print` і поверненням значень через `return`, правилом пошуку змінних LEGB (Local $\rightarrow$ Enclosing $\rightarrow$ Global $\rightarrow$ Built-in), а також використанням docstrings (`__doc__`) та анотацій типів (`__annotations__`); у третьому завданні я створила бібліотеку з чотирьох функцій для обробки прізвища з ініціалами, підрахунком символів за допомогою `.lower()` та розворотом рядка циклом, а в четвертому — побудувала повністю модульну програму для розрахунку балів, де кожна функція відповідає лише за одну задачу, а весь сценарій керується через `main()`, що забезпечує перевикористання коду, просту підтримку та високу читабельність.

```

#Батури Каріни, ІТ - 31
def read_grade(prompt: str) -> int:
    """Запитує в користувача оцінку, доки не буде введено ціле число від 0 до 100"""
    while True:
        value = input(prompt)
        if not value.isdigit():
            print("Помилка: вводити можна лише цифри")
            continue
        grade = int(value)
        if 0 <= grade <= 100:
            return grade
        print("Помилка: значення має бути від 0 до 100")

def to_letter(grade: float) -> str:
    """Перетворює числову оцінку в буквену (A-F), використовуючи декілька return."""
    if grade >= 90:
        return "A"
    if grade >= 82:
        return "B"
    if grade >= 75:
        return "C"
    if grade >= 64:
        return "D"
    if grade >= 60:
        return "E"
    return "F"

def average(grades: list[int]) -> float:
    """Повертає середнє арифметичне для списку оцінок."""
    return sum(grades) / len(grades)

def count_above(grades: list[int], limit: float) -> int:
    """Повертає кількість оцінок, які чітко більші за вказаний ліміт."""

```

```

    """Повертає кількість оцінок, які чітко більші за вказаний ліміт."""
    count = 0
    for g in grades:
        if g > limit:
            count += 1
    return count

def print_report(name: str, group: str, grades: list[int]) -> None:
    """Виводить відформатований звіт для студента за допомогою допоміжних функцій"""
    avg_grade = average(grades)
    letter_grade = to_letter(avg_grade)
    above_avg_count = count_above(grades, avg_grade)
    grades_str = " ".join(str(g) for g in grades)

    print("--- Звіт ---")
    print(f"Студент: {name}, група {group}")
    print(f"Оцінки: {grades_str}")
    print(f"Середній бал: {avg_grade:.2f} -> {letter_grade}")
    print(f"Найкращий: {max(grades)}, найгірший: {min(grades)}")
    print(f"Вище середнього: {above_avg_count}")

def main() -> None:
    """Головна точка входу в програму."""
    name = "Каріна Батура"
    group = "ІТ-31"

    first_name = "Каріна"
    n = max(len(first_name), 3)

    print(f"{name}, {group}")

    grades = []
    for i in range(1, n + 1):
        grade = read_grade(f"Оцінка {i} (0-100): ")

```

```

PS C:\Users\Windows 11 Pro\OneDrive\Desktop\practic_5> & "C:\Python\Python313\python.exe" "c:/Users/windows 11 Pro/OneDrive/Desktop/practic_5.py"
Каріна Батура, ІТ-31
Оцінка 1 (0-100): 45
Оцінка 2 (0-100): 35
Оцінка 3 (0-100): 60
Оцінка 4 (0-100): 80
Оцінка 5 (0-100): 90
Оцінка 6 (0-100): 679
Помилка: значення має бути від 0 до 100
Оцінка 6 (0-100): 56
--- Звіт ---
Студент: Каріна Батура, група ІТ-31
Оцінки: 45 35 60 80 90 56
Середній бал: 61.00 -> E
Найкращий: 90, найгірший: 35
Вище середнього: 2
PS C:\Users\Windows 11 Pro\OneDrive\Desktop\practic_5>

```